

# ЛАБОРАТОРНАЯ РАБОТА №4: «Автоматизация задач ИБ»

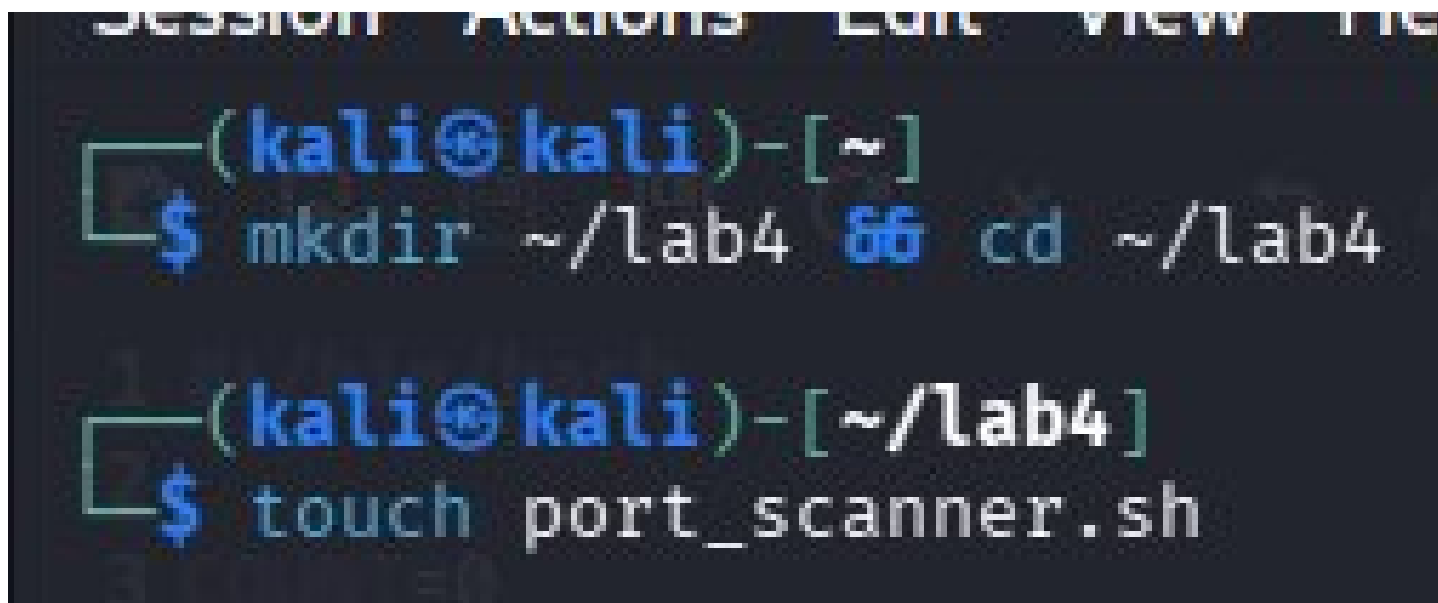
Легенда: ты — специалист по безопасности, которому нужно автоматизировать рутинные задачи. Твоя цель — написать три скрипта для аудита системы, сканирования и мониторинга.

## Задача 1. Скрипт для автоматического сканирования портов

**Шаг 1.1.** Создай скрипт port\_scanner.sh, который:

- Принимает IP-адрес или хост как аргумент командной строки
- Проверяет, передан ли аргумент (если нет — выводит сообщение об ошибке и инструкцию по использованию)
- Сканирует стандартные порты (22, 80, 443, 8080, 3306) на целевом хосте
- Для каждого открытого порта выводит сообщение: [+] Порт PORT открыт
- Если ни один порт не открыт, выводит: [-] Нет открытых портов

создали папку для лабы и файл port\_scanner.sh



```
SESSION ACTIONS EDIT VIEW FILE
(kali@kali)-[~]
$ mkdir ~/lab4 && cd ~/lab4

(kali@kali)-[~/lab4]
$ touch port_scanner.sh
```

далее я написала скрипт, вот такой получился, полностью соответствует ТЗ

```
1 #!/bin/bash
2
3 COUNT=0
4
5 PORTS=(
6     "22"
7     "80"
8     "443"
9     "8080"
10    "3306"
11 )
12
13 read -p "enter the IP:" IP
14 if [ -z "$IP" ]; then
15     echo "ERROR"
16     exit 1
17 fi
18
19 for PORT in "${PORTS[@]}"; do
20     if nc -zv -w 2 "$IP" "$PORT" &>/dev/null; then
21         echo "[+] PORT $PORT is open"
22         ((COUNT++))
23     fi
24 done
25 if [ $COUNT -eq 0 ]; then
26     echo "None of ports is open"
27 fi
28 |
```

далее сделали файл исполняемым

```
11
12 (kali@kali)-[~/lab4]
13 $ chmod +x port_scanner.sh
```

и далее проверили работу скрипта

```
(kali@kali)-[~/lab4]
$ ./port_scanner.sh
enter the IP:192.168.1.1
None of ports is open
```

```
(kali@kali)-[~/lab4]
$ ./port_scanner.sh
enter the IP:10.0.0.1
None of ports is open
```

**Шаг 1.2.** Добавь в скрипт функцию `scan_port()`, которая:

- Принимает IP и порт как аргументы
- Проверяет доступность порта (используй `nc -z -w 1 IP PORT` или `timeout 1 bash -c "echo > /dev/tcp/IP/PORT" 2>/dev/null`)
- Возвращает 0, если порт открыт, и 1 если закрыт

поменяли немного внешний вид под новые требования, но суть осталась той же

```
1 #!/bin/bash
2
3 COUNT=0
4
5 PORTS=(
6     "22"
7     "80"
8     "443"
9     "8080"
10    "3306"
11 )
12
13 scan_port(){
14     if nc =zv -w 2 "$1" "$2" &>/dev/null; then
15         echo "[+] PORT $2 is open"
16         return 0
17     else
18         echo "[-] PORT $2 is closed"
19         return 1
20     fi
21 }
22
23 read -p "enter the IP:" IP
24 if [ -z "$IP" ]; then
25     echo "ERROR"
26     exit 1
27 fi
28
29 for PORT in "${PORTS[@]}; do
30     if scan_port "$IP" "$PORT"; then
31         ((COUNT++))
32     fi
33 done
34
35 if [ $COUNT -eq 0 ]; then
36     echo "None of ports is open"
37 fi
38 |
```

и получили уже более подробный вывод

```
(kali@kali)-[~/lab4]
$ ./port_scanner.sh
enter the IP:192.168.1.1
[-] PORT 22 is closed
[-] PORT 80 is closed
[-] PORT 443 is closed
[-] PORT 8080 is closed
[-] PORT 3306 is closed
None of ports is open
```

**Шаг 1.3.** Добавь логирование:

- Создай функцию `log_message()`, которая выводит сообщения с временной меткой в формате `[YYYY-MM-DD HH:MM:SS] MESSAGE`
- Используй эту функцию для всех выводимых сообщений

снова приукрасили скрипт, вот так получилось

```

1  #!/bin/bash
2
3  COUNT=0
4
5  PORTS=(
6      "22"
7      "80"
8      "443"
9      "8080"
10     "3306"
11 )
12
13 log_message(){
14     echo "$(date '+%Y-%m-%d %H:%M:%S') "
15 }
16
17 scan_port(){
18     if nc =zv -w 2 "$1" "$2" &>/dev/null; then
19         echo "$(log_message) [+] PORT $2 is open"
20         return 0
21     else
22         echo "$(log_message) [-] PORT $2 is closed"
23         return 1
24     fi
25 }
26
27 read -p "enter the IP:" IP
28 if [ -z "$IP" ]; then
29     echo "ERROR"
30     exit 1
31 fi
32
33 for PORT in "${PORTS[@]}; do
34     if scan_port "$IP" "$PORT"; then
35         ((COUNT++))
36     fi
37 done
38
39 if [ $COUNT -eq 0 ]; then
40     echo "None of ports is open"
41 fi

```

вот получили такой вывод теперь

```
(kali㉿kali)-[~/lab4]  
$ ./port_scanner.sh  
enter the IP:192.168.1.1  
2026-07-18 17:26:44 [-] PORT 22 is closed  
2026-07-18 17:26:44 [-] PORT 80 is closed  
2026-07-18 17:26:44 [-] PORT 443 is closed  
2026-07-18 17:26:44 [-] PORT 8080 is closed  
2026-07-18 17:26:44 [-] PORT 3306 is closed  
None of ports is open
```

**Шаг 1.4.** Протестируй скрипт:

- Запусти сканирование localhost (127.0.0.1)

```
(kali㉿kali)-[~/lab4]  
$ ./port_scanner.sh  
enter the IP:127.0.0.1  
2026-07-19 03:46:40 [+] PORT 22 is open  
2026-07-19 03:46:40 [-] PORT 80 is closed  
2026-07-19 03:46:40 [-] PORT 443 is closed  
2026-07-19 03:46:40 [-] PORT 8080 is closed  
2026-07-19 03:46:40 [-] PORT 3306 is closed
```

- Запусти сканирование без аргументов (должна быть ошибка)

```
(kali@kali)-[~/lab4]
$ ./port_scanner.sh
enter the IP:127.0.0.1
2026-07-18 17:34:30 [-] PORT is closed
2026-07-18 17:34:30 [-] PORT is closed
2026-07-18 17:34:30 [-] PORT is closed
2026-07-18 17:34:30 [-] PORT is closed
2026-07-18 17:34:30 [-] PORT is closed
None of ports is open
```

изначально я скрипт не меняла и просто пропали номера портов, но, очевидно, скрипт не сработал, но ошибку не выбило. поэтому я решила в начало скрипта после шибанга добавить set -euo, чтобы ошибку прям написало мне. и вот что получилось:

```
(kali@kali)-[~/lab4]
$ ./port_scanner.sh
enter the IP:127.0.0.1
./port_scanner.sh: line 20: $1: unbound variable
```

- Запусти сканирование какого-нибудь внешнего хоста (например, google.com)

мой скрипт работает и для адресов и для доменов благодаря нашей утилите netstat:

```
(kali@kali)-[~/lab4]
$ ./port_scanner.sh
enter the IP:google.com
2026-07-19 03:50:02 [-] PORT 22 is closed
2026-07-19 03:50:02 [+] PORT 80 is open
2026-07-19 03:50:02 [+] PORT 443 is open
2026-07-19 03:50:05 [-] PORT 8080 is closed
2026-07-19 03:50:07 [-] PORT 3306 is closed
```

получаем открытые порты 80 и 443 - это HTTP и HTTPS соответственно. ну и правильно, это же доменное имя браузера, соответственно порты и открыты.



## Задача 2. Скрипт для парсинга логов и алертинга

**Шаг 2.1.** Создай скрипт `log_analyzer.sh`, который:

- Принимает путь к лог-файлу как аргумент
- Проверяет, существует ли файл (если нет — выводит ошибку и выходит)
- Анализирует файл `/var/log/auth.log` (или указанный файл) на наличие:
  - Неудачных попыток входа (строки с "Failed password")
  - Успешных входов (строки с "Accepted password")
  - Использования `sudo` (строки с "sudo")

мы создали новый файл `log_analyzer.sh`

затем написали скрипт небольшой, который проверяет существование файла с логами ну и затем извлекает каждую строчку, где есть попытки входа удачные/неудачные и использования `sudo` при помощи команды `grep`

```
1 #!/bin/bash
2
3 read -p "enter the path to log file: " path
4
5 if [ -e "$path" ]; then
6     echo "file exists!"
7 else
8     echo "ERROR file doesnt exit"
9     exit 1
10 fi
11
12 grep -E "Failed password|Accepted password|sudo" $path
13 |
```

затем наложили права на исполнение

```
(kali@kali)-[~/lab4]
$ touch log_analyzer.sh

(kali@kali)-[~/lab4]
$ chmod +x log_analyzer.sh
```

и получили такой вывод, действительно, скрипт сработал верно

```
(kali@kali)-[~/lab4]
$ ./log_analyzer.sh
enter the path to log file: /home/kali/incident_lab/test_auth.log
file exists!
Jul 19 10:15:23 kali sshd[1234]: Failed password for invalid user admin from 192.168.1.100 port 54321 ssh2
Jul 19 10:15:25 kali sshd[1234]: Failed password for invalid user admin from 192.168.1.100 port 54322 ssh2
Jul 19 10:15:27 kali sshd[1234]: Failed password for invalid user root from 192.168.1.100 port 54323 ssh2
Jul 19 10:15:29 kali sshd[1234]: Failed password for invalid user test from 192.168.1.100 port 54324 ssh2
Jul 19 10:15:31 kali sshd[1234]: Failed password for invalid user admin from 192.168.1.100 port 54325 ssh2
Jul 19 10:15:33 kali sshd[1234]: Failed password for invalid user admin from 192.168.1.100 port 54326 ssh2
Jul 19 10:16:01 kali sshd[1235]: Accepted password for kali from 192.168.1.50 port 45678 ssh2
Jul 19 10:17:45 kali sudo: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/bin/ls /root
Jul 19 10:18:12 kali sshd[1236]: Failed password for invalid user oracle from 10.0.0.55 port 33445 ssh2
Jul 19 10:18:14 kali sshd[1236]: Failed password for invalid user oracle from 10.0.0.55 port 33446 ssh2
Jul 19 10:18:16 kali sshd[1236]: Failed password for invalid user postgres from 10.0.0.55 port 33447 ssh2
Jul 19 10:19:30 kali sshd[1237]: Accepted password for kali from 192.168.1.50 port 45680 ssh2
Jul 19 10:20:15 kali sudo: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/apt update
Jul 19 10:21:00 kali sshd[1238]: Failed password for invalid user admin from 172.16.0.99 port 22334 ssh2
Jul 19 10:21:02 kali sshd[1238]: Failed password for invalid user admin from 172.16.0.99 port 22335 ssh2
Jul 19 10:21:04 kali sshd[1238]: Failed password for invalid user root from 172.16.0.99 port 22336 ssh2
Jul 19 10:21:06 kali sshd[1238]: Failed password for invalid user test from 172.16.0.99 port 22337 ssh2
Jul 19 10:21:08 kali sshd[1238]: Failed password for invalid user guest from 172.16.0.99 port 22338 ssh2
Jul 19 10:21:10 kali sshd[1238]: Failed password for invalid user admin from 172.16.0.99 port 22339 ssh2
Jul 19 10:21:12 kali sshd[1238]: Failed password for invalid user admin from 172.16.0.99 port 22340 ssh2
Jul 19 10:22:00 kali sudo: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/bin/cat /etc/shadow
```

**Шаг 2.2.** Для каждого типа событий:

- Подсчитай общее количество

в строку с `grep` добавили флаг `-c`, он поможет посчитать общее количество подходящих строк

получили такой вывод

```
(kali@kali)-[~/lab4]
$ ./log_analyzer.sh
enter the path to log file: /home/kali/incident_lab/test_auth.log
file exists!
21
```

итого 21 подходящая строка

- Выведи топ-3 IP-адреса, с которых было больше всего попыток (используй `awk`, `sort`, `uniq -c`, `sort -rn`, `head -3`)

вот у нас получилась такая цепочка `grep -E "Failed password|Accepted password|sudo" $path | grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" | sort | uniq -c | sort -rn | head -3`

рассказываю по очереди:

`grep -E "Failed password|Accepted password|sudo" $path` - это я поясняла уже вроде с того задания

`grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}"` - тут получается выбирается благодаря флагу `-o` (only matching) только то, что четко совпало с шаблоном (макетом адреса, написанного далее). то есть мы оставили не все слова с каждой строки, а просто выцепили адрес

`sort` - далее сортируем наши адреса, чтобы в дальнейшем `uniq` справился четко

`uniq -c` - посчитывает количество дубликатов

`sort -rn` - снова сортируем, но уже по количеству повторений (`-n`) и в обратном порядке (от большего к меньшему)

`head -3` - выводим первые три записи

- Сохрани результаты в отчетный файл `security_report.txt`

чтобы сохранить, просто в конец строки добавили `> security_report.txt`

**Шаг 2.3.** Добавь функцию `generate_alert()`, которая:

- Принимает уровень опасности (INFO, WARNING, CRITICAL) и сообщение
- Если количество неудачных попыток входа  $> 10$  — выводит CRITICAL алерт
- Если количество неудачных попыток  $> 5$  — выводит WARNING алерт
- Иначе выводит INFO алерт

я написала функцию

```

1 #!/bin/bash
2
3 generate_alert(){
4     local level=$1
5     local message=$2
6     local count=$3
7
8     if [ "$count" -gt 10 ]; then
9         echo "CRITICAL alert: $message ($count attempts)"
10    elif [ "$count" -gt 5 ]; then
11        echo "WARNING alert: $message ($count attempts)"
12    else
13        echo "INFO alert: $message ($count attempts)"
14    fi
15 }
16
17 read -p "enter the path to log file: " path
18
19 if [ -e "$path" ]; then
20     echo "file exists!" > security_report.txt
21 else
22     echo "ERROR file doesnt exist" > security_report.txt
23     exit 1
24 fi
25
26 grep -E "Failed password|Accepted password|sudo" $path | grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" | sort | uniq -c | sort -rn | head -3 >> security_report.txt
27
28 |

```

она принимает на вход уровень опасности, сообщение и количество попыток неудачного ввода. но мне кажется, что уровень опасности здесь на прием это лишнее, ведь мы используем только количество и сообщение. поэтому я наверное строку убегу, оставлю только два аргумента

```

1 #!/bin/bash
2
3 generate_alert(){
4     local message=$1
5     local count=$2
6
7     if [ "$count" -gt 10 ]; then
8         echo "CRITICAL alert: $message ($count attempts)"
9     elif [ "$count" -gt 5 ]; then
10        echo "WARNING alert: $message ($count attempts)"
11    else
12        echo "INFO alert: $message ($count attempts)"
13    fi
14 }
15
16 read -p "enter the path to log file: " path
17
18 if [ -e "$path" ]; then
19     echo "file exists!" > security_report.txt
20 else
21     echo "ERROR file doesnt exist" > security_report.txt
22     exit 1
23 fi
24
25 grep -E "Failed password|Accepted password|sudo" $path | grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" | sort | uniq -c | sort -rn | head -3 >> security_report.txt
26
27 failed_count=$(grep -c "Failed password" "$path")
28 generate_alert "Brute force trying" "$failed_count"
29

```

вот такой итоговый скрипт получился

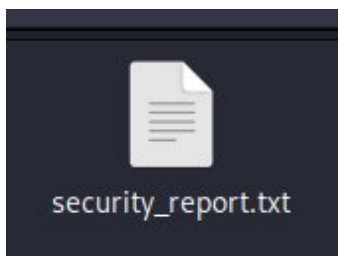
## Шаг 2.4. Протестируй скрипт:

- Создай тестовый лог-файл с несколькими строками "Failed password" и "Accepted password"
- Запусти скрипт на этом файле
- Проверь, что отчет security\_report.txt создан и содержит правильную информацию

запуск, все выводы я решила запихнуть в security\_report.txt

```
(kali@kali)-[~/lab4]  
$ ./log_analyzer.sh  
enter the path to log file: /home/kali/incident_lab/test_auth.log
```

файл действительно создан



вот его содержимое

```
1 file exists!  
2      7 172.16.0.99  
3      6 192.168.1.100  
4      3 10.0.0.55  
5 CRITICAL alert: Bruite force trying (16 attempts)  
6
```

итак, судя по выводу видим, что Файл с логами действительно существует он был искусственно создан. Топ 3 IP адреса по попыткам доступа видим и количество запросов с каждого ip-адреса. ну и последнее сообщение критическая попытка брутфорс атаки 16 попыток.

### Задача 3. Инструмент для бэкапа с ротацией

**Шаг 3.1.** Создай скрипт backup\_tool.sh, который:

- Принимает два аргумента: путь к директории для бэкапа и путь для хранения бэкапов
- Проверяет, что оба аргумента переданы
- Проверяет, что директория для бэкапа существует
- Создает архив с именем backup\_YYYY-MM-DD\_HH-MM-SS.tar.gz (используй tar -czf)

создали файл и дали права на выполнение



```
(kali@kali)-[~/lab4]
$ touch backup_tool.sh

(kali@kali)-[~/lab4]
$ chmod +x backup_tool.sh
```

написали скрипт, полностью соответствующий ТЗ

```
1 #!/bin/bash
2
3 read -p "enter the path to target directory" target
4 read -p "enter the path to create a backup directory" storage
5
6 if [ -z target ] || [ -z storage ]; then
7     echo "empty input"
8     exit 1
9 fi
10
11 if [ -e "$target" ]; then
12     echo "target directory for backup exists"
13     if [ ! -e "$storage" ]; then
14         mkdir -p "$storage"
15     fi
16     BACKUP_NAME="backup_$(date +%Y-%m-%d).tar.gz"
17     tar -czf "$BACKUP_NAME" "$target"
18     cp -a "$BACKUP_NAME" "$storage/"
19 else
20     echo "target directory doesnt exist"
21     exit 1
22 fi
23
24
```

**Шаг 3.2.** Реализуй ротацию бэкапов:

- Храни только последние 5 бэкапов
- После создания нового бэкапа удаляй старые (используй `ls -t` для сортировки по времени и `tail -n +6` для получения всех кроме первых 5)

```

1 #!/bin/bash
2
3 check_disc_space(){
4
5 }
6
7 read -p "enter the path to target directory" target
8 read -p "enter the path to create a backup directory" storage
9
10 if [ -z target ] || [ -z storage ]; then
11     echo "empty input"
12     exit 1
13 fi
14
15 if [ -e "$target" ]; then
16     echo "target directory for backup exists"
17     if [ ! -e "$storage" ]; then
18         mkdir -p "$storage"
19     fi
20     BACKUP_NAME="backup_$(date +%Y-%m-%d).tar.gz"
21     tar -czf "$BACKUP_NAME" "$target"
22     cp -a "$BACKUP_NAME" "$storage/"
23 else
24     echo "target directory doesnt exist"
25     exit 1
26 fi
27
28 line_count=$(cd "$storage" | ls -1 -A | wc -l)
29
30 if [ "$line_count" -gt 5 ]; then
31     ls -t | tail -n +6 | xargs -d '\n' rm -f
32 fi |

```

что добавили: сначала мы нашли количество файлов внутри нашего storage. затем условие если количество файлов больше 5, то мы их сортируем по времени, извлекаем все файлы, кроме последних 5, при помощи xargs превращаем их в аргументы и передаем команде rm для удаления.

**Шаг 3.3.** Добавь функцию check\_disk\_space(), которая:

- Проверяет свободное место на диске (используй df)
- Если свободного места меньше 100MB — выводит WARNING и не создает бэкап
- Иначе продолжает работу

```

1 #!/bin/bash
2
3 check_disc_space(){
4     local free_space=$(df -m / | awk 'NR==2 {print $4}')
5     if ["$free_space" -lt 100]; then
6         echo "WARNING! No free space, $free_space left"
7         return 1
8     else
9         echo "OK! $free_space left, CONTINUE"
10        return 0
11    fi
12 }
13
14 read -p "enter the path to target directory" target
15 read -p "enter the path to create a backup directory" storage
16
17 if [ -z target ] || [ -z storage ]; then
18     echo "empty input"
19     exit 1
20 fi
21
22 if [ -e "$target" ]; then
23     echo "target directory for backup exists"
24     if [ ! -e "$storage" ]; then
25         mkdir -p "$storage"
26     fi
27     BACKUP_NAME="backup_$(date +%Y-%m-%d).tar.gz"
28     tar -czf "$BACKUP_NAME" "$target"
29     cp -a "$BACKUP_NAME" "$storage/"
30 else
31     echo "target directory doesnt exist"
32     exit 1
33 fi
34
35 line_count=$(cd "$storage" | ls -1 -A | wc -l)
36
37 if [ "$line_count" -gt 5 ]; then
38     ls -t | tail -n +6 | xargs -d '\n' rm -f
39 fi
40

```

была добавлена функция `check_disc_space()`, она проверяет при помощи команды `df -m /` свободное место на диске, флаг `-m` убирает единицу измерения и делает цифры понятными. а `/` берет корневой диск, ну то есть все диски вместе. если места остается меньше 100, то мы выводим соответствующее сообщение и кидаем код ошибки (1)

**Шаг 3.4.** Добавь логирование всех действий:

- Создание бэкапа
- Удаление старых бэкапов
- Ошибки (если директория не найдена, не хватает места и т.д.)



```

1 #!/bin/bash
2
3 check_disc_space(){
4     local free_space=$(df -m / | awk 'NR==2 {print $4}')
5     if ["$free_space" -lt 100]; then
6         echo "WARNING! No free space, $free_space left"
7         return 1
8     else
9         echo "OK! $free_space left, CONTINUE"
10        return 0
11    fi
12 }
13
14 log_message(){
15     local type=$1
16     local message=$2
17     echo "$(date '+%Y-%m-%d %H:%M:%S') $type $message" >> "$LOG_FILE"
18 }
19
20 read -p "enter the path to target directory" target
21 read -p "enter the path to create a backup directory" storage
22
23 if [ -z target ] || [ -z storage ]; then
24     echo "empty input"
25     exit 1
26 fi
27
28 if [ -e "$target" ]; then
29     echo "target directory for backup exists"
30     if [ ! -e "$storage" ]; then
31         mkdir -p "$storage"
32     fi
33     BACKUP_NAME="backup_$(date +%Y-%m-%d).tar.gz"
34     tar -czf "$BACKUP_NAME" "$target"
35     cp -a "$BACKUP_NAME" "$storage/"
36 else
37     echo "target directory doesnt exist"
38     exit 1
39 fi
40
41 line_count=$(cd "$storage" | ls -1 -A | wc -l)
42
43 if [ "$line_count" -gt 5 ]; then
44     ls -t | tail -n +6 | xargs -d '\n' rm -f
45 fi
46

```

добавлена функция `log_message()`, которая принимает на вход тип лога (ERROR, WARNING, INFO) и само сообщение и превращает это в полноценный лог: добавляет время и дату, потом пишет тип лога и затем само сообщение и в конце концов сохраняет в переменную `LOG_FILE`.

**Шаг 3.5.** Протестируй скрипт:

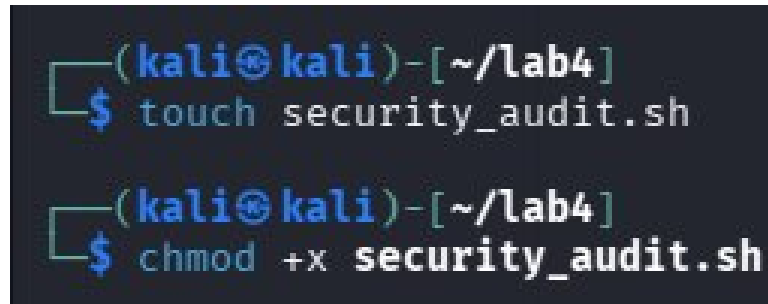
- Создай тестовую директорию с несколькими файлами
- Запусти скрипт для создания бэкапа
- Запусти скрипт несколько раз, чтобы проверить ротацию (должно остаться только 5 последних бэкапов)

**Задача 4. 🔥 Бонус: Создание полноценного инструмента аудита**

**Шаг 4.1.** Объедини все предыдущие навыки и создай скрипт `security_audit.sh`, который:

- Проверяет наличие SUID-файлов (как в Лабораторной 1)
- Проверяет подозрительные процессы (как в Лабораторной 2)
- Проверяет открытые порты (как в Лабораторной 3)
- Проверяет cron-задачи
- Генерирует полный отчет в файл `audit_report.txt`

начинаем по базе - создание и права на исполнение

A screenshot of a terminal window with a dark background. The prompt is `(kali@kali)-[~/lab4]`. The first command entered is `$ touch security_audit.sh`. The second command entered is `$ chmod +x security_audit.sh`.

```
(kali@kali)-[~/lab4]
$ touch security_audit.sh

(kali@kali)-[~/lab4]
$ chmod +x security_audit.sh
```

**Шаг 4.2.** Добавь цветное оформление вывода:

- Зеленый цвет для OK
- Желтый цвет для WARNING
- Красный цвет для CRITICAL

**Шаг 4.3.** Добавь возможность выбора режима:

- `./security_audit.sh --quick` — быстрая проверка (только критичные вещи)
- `./security_audit.sh --full` — полная проверка (всё)
- `./security_audit.sh --help` — справка по использованию

короче 4.2 и 4.3 делать нет желания ахахахах, я сделала 4.1

```
1 #!/bin/bash
2
3 touch "report.txt"
4
5 read -p "enter path to target directory" $path
6
7 echo "SECURITY REPORT $(date '+%Y-%m-%d %H:%M:%S')" > "report.txt"
8 echo "" >> "report.txt"
9
10 echo "1. search SUID bits" >> "report.txt"
11 echo "" >> "report.txt"
12 find "$path" -perm -4000 -type -f 2>/dev/null >> "report.txt"
13 echo "" >> "report.txt"
14
15 echo "2. suspicious processes" >> "report.txt"
16 echo "" >> "report.txt"
17 echo "processes with most used cpu:" >> "report.txt"
18 echo "" >> "report.txt"
19 ps aux --sort=-%cpu | head -n -10 >> "report.txt"
20 echo "" >> "report.txt"
21
22 echo "3. opened ports" >> "report.txt"
23 echo "" >> "report.txt"
24 ss -tulpn >> "report.txt" 2>/dev/null
25 echo "" >> "report.txt"
26
27 echo "4. cron tasks" >> "report.txt"
28 echo "" >> "report.txt"
29 echo "user tasks" >> "report.txt"
30 crontab -l >> "report.txt" 2>/dev/null || echo "no user tasks" >> "report.txt"
31 echo "" >> "report.txt"
32 echo "system tasks" >> "report.txt"
33 cat /etc/crontab 2>/dev/null >> "report.txt"
34 echo "" >> "report.txt"
35
36 echo "audit is finished" >> "report.txt"
37 |
```

вот такой получился скрипт

а вот такой вывод (report.txt)

```
1|
2 system tasks
3 # /etc/crontab: system-wide crontab
4 # Unlike any other crontab you don't have to run the `crontab'
5 # command to install the new version when you edit this file
6 # and files in /etc/cron.d. These files also have username fields,
7 # that none of the other crontabs do.
8
9 SHELL=/bin/sh
10 PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
11
12 # Example of job definition:
13 # ._____ minute (0 - 59)
14 # | ._____ hour (0 - 23)
15 # | | ._____ day of month (1 - 31)
16 # | | | ._____ month (1 - 12) OR jan,feb,mar,apr ...
17 # | | | | ._____ day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
18 # | | | | |
19 # * * * * * user-name command to be executed
20 17 * * * * root cd / && run-parts --report /etc/cron.hourly
21 25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
22 47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
23 52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
24 #
25
26 audit is finished
27
```